

[Back to Home](#)

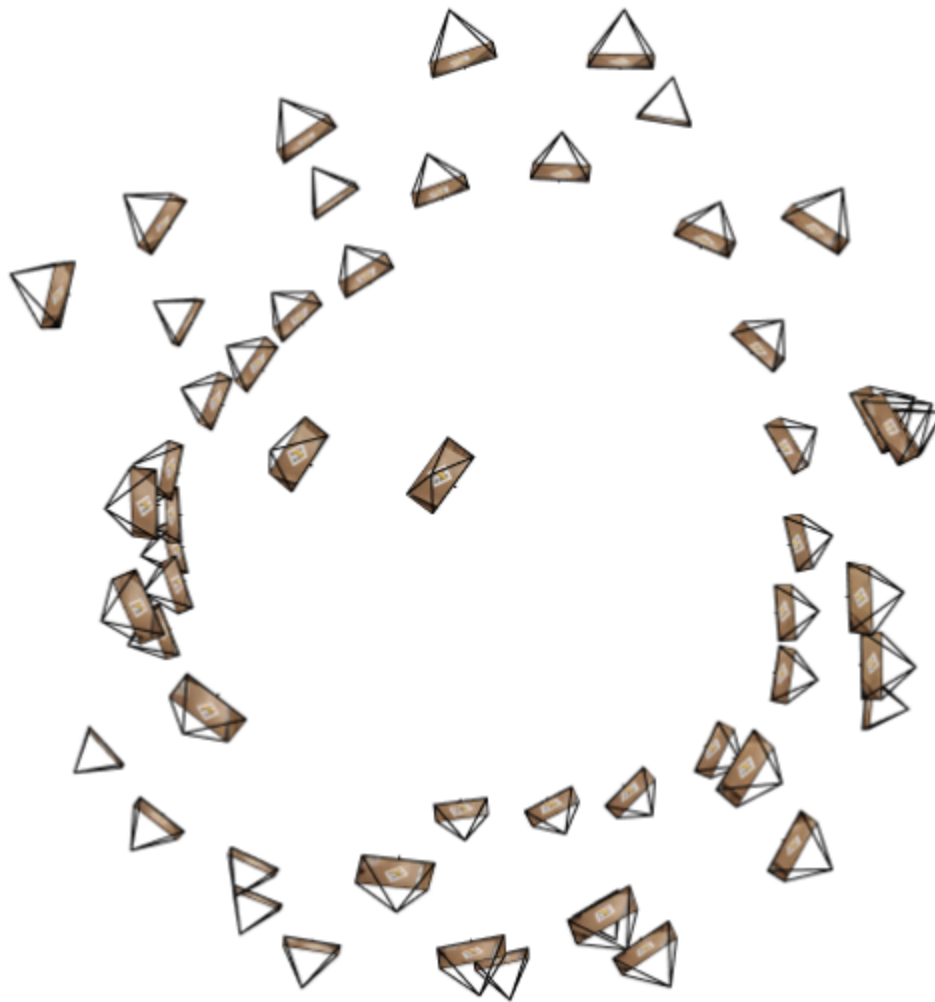
Project 5: Neural Radiance Fields (NeRF)

Part 0: Camera Calibration and 3D Scanning

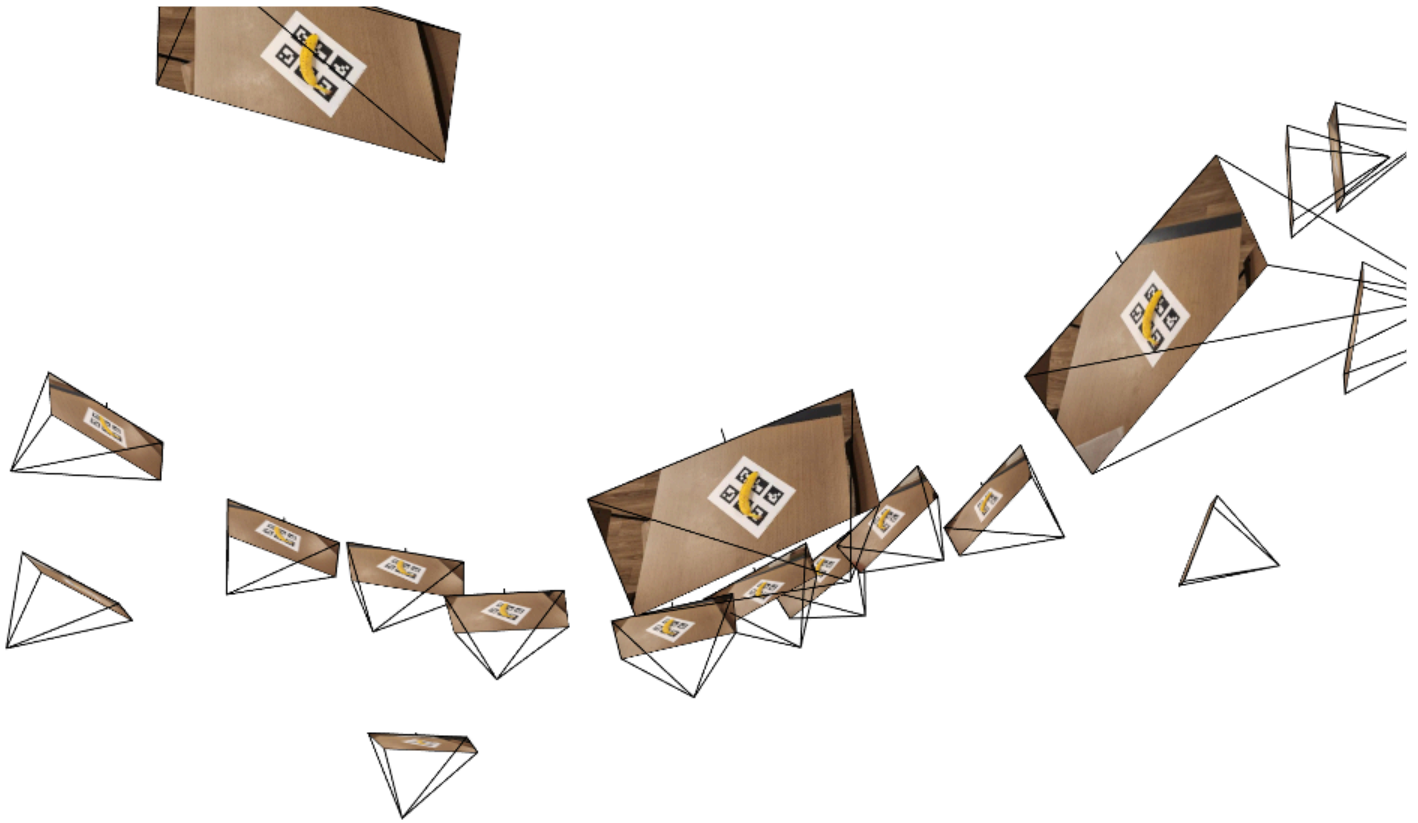
Overview

I captured 30-50 images of ArUco tags for camera calibration using consistent zoom and lighting. The calibration process used `cv2.calibrateCamera()` to compute the intrinsic matrix and distortion coefficients. For object scanning, I photographed my object from various angles and distances.

Camera Pose Visualization



Camera frustum visualization with projected images (View 1)



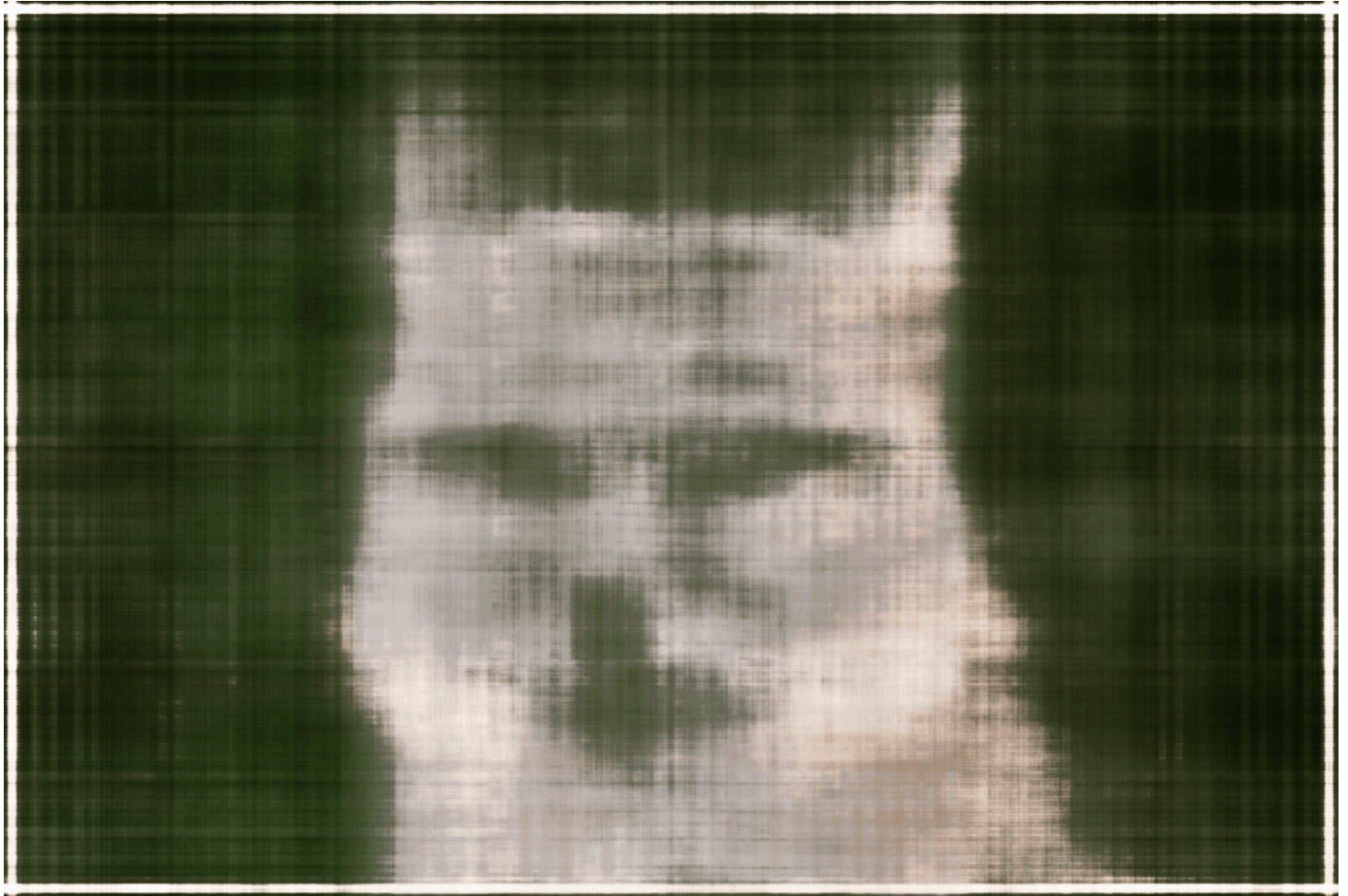
Camera frustum visualization with projected images (View 2)

Part 1: 2D Neural Field Fitting

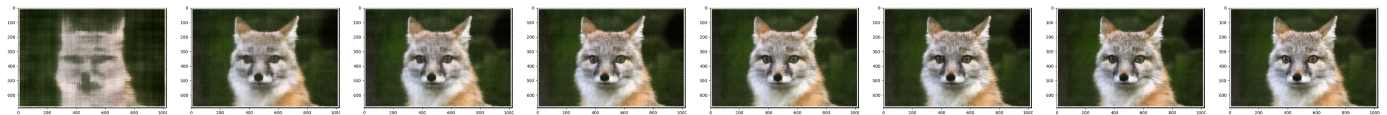
Network Architecture

I implemented an MLP with positional encoding to map 2D pixel coordinates to RGB colors. The network uses fully connected layers with ReLU activations and positional encoding with $L=10$ frequency levels. Training used Adam optimizer with learning rate $1e-2$.

Training Progression



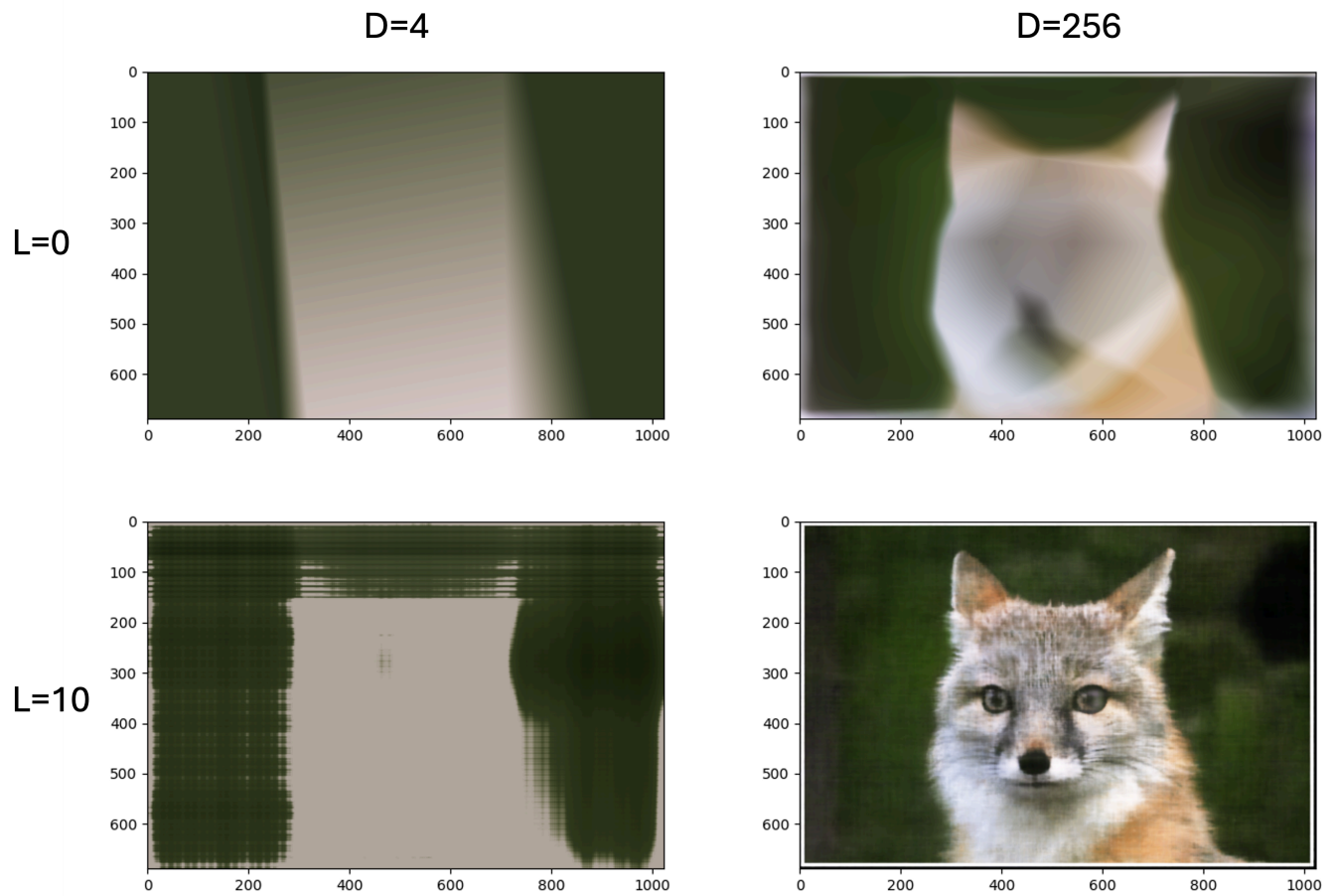
Animated training progression showing how the network learns the image



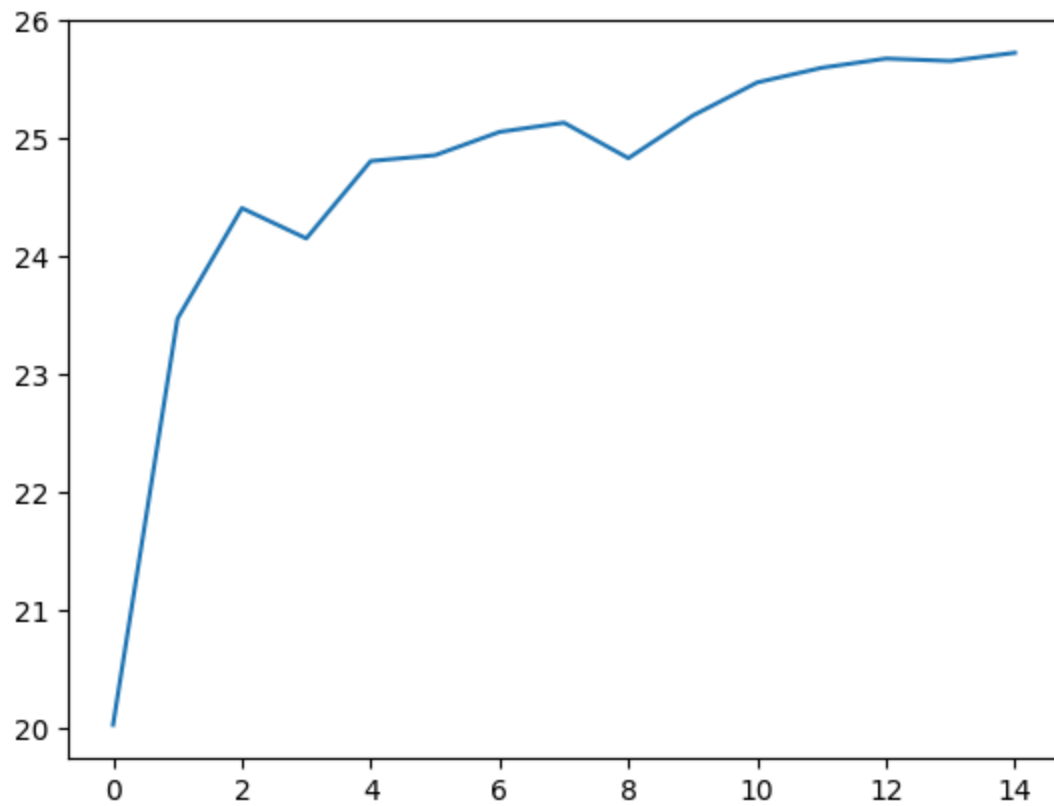
Training progression at different iterations

Hyperparameter Exploration

I tested four different configurations by varying positional encoding (PE) frequency levels ($L = 0$ vs. $L = 10$) and network hidden layer width (4 vs. 256). Networks with higher PE frequency ($L = 10$) can capture fine-grained image details more effectively than those with $L = 0$. Similarly, wider networks (width = 256) offer greater representational capacity for learning complex color distributions compared to narrow networks (width = 4).



Results with different network configurations and PE frequencies

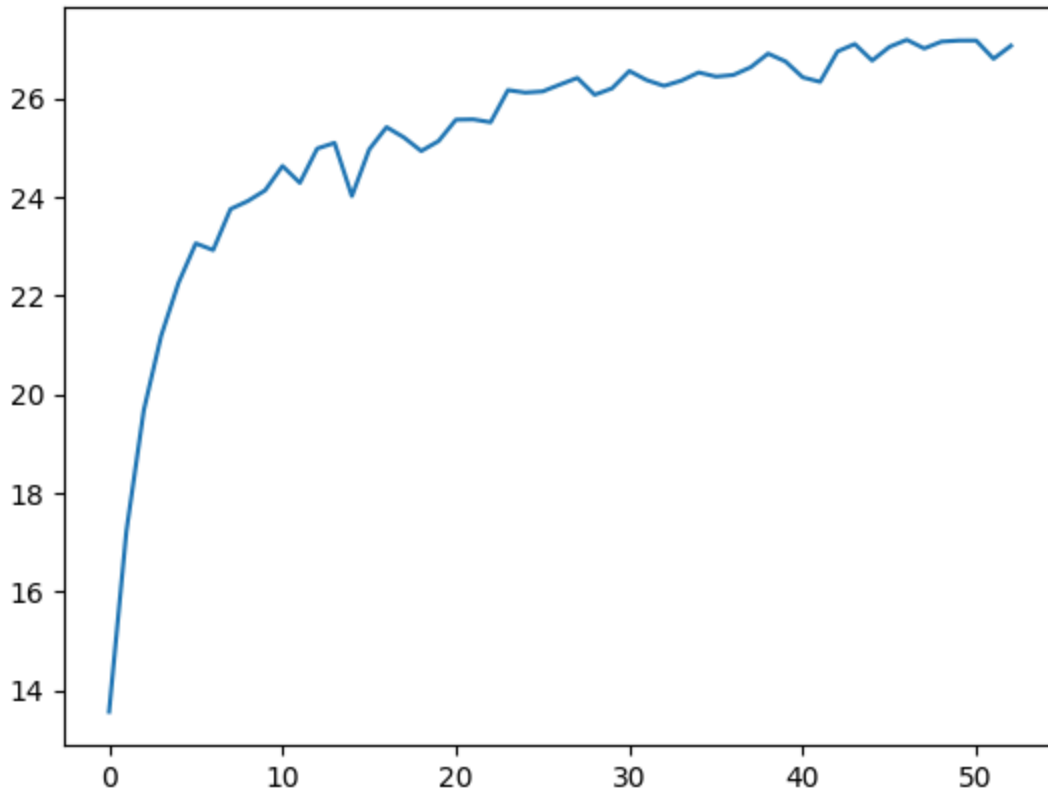


PSNR over training iterations

Custom Images - Marugame



Training progression for Marugame image



PSNR over training iterations for Marugame image

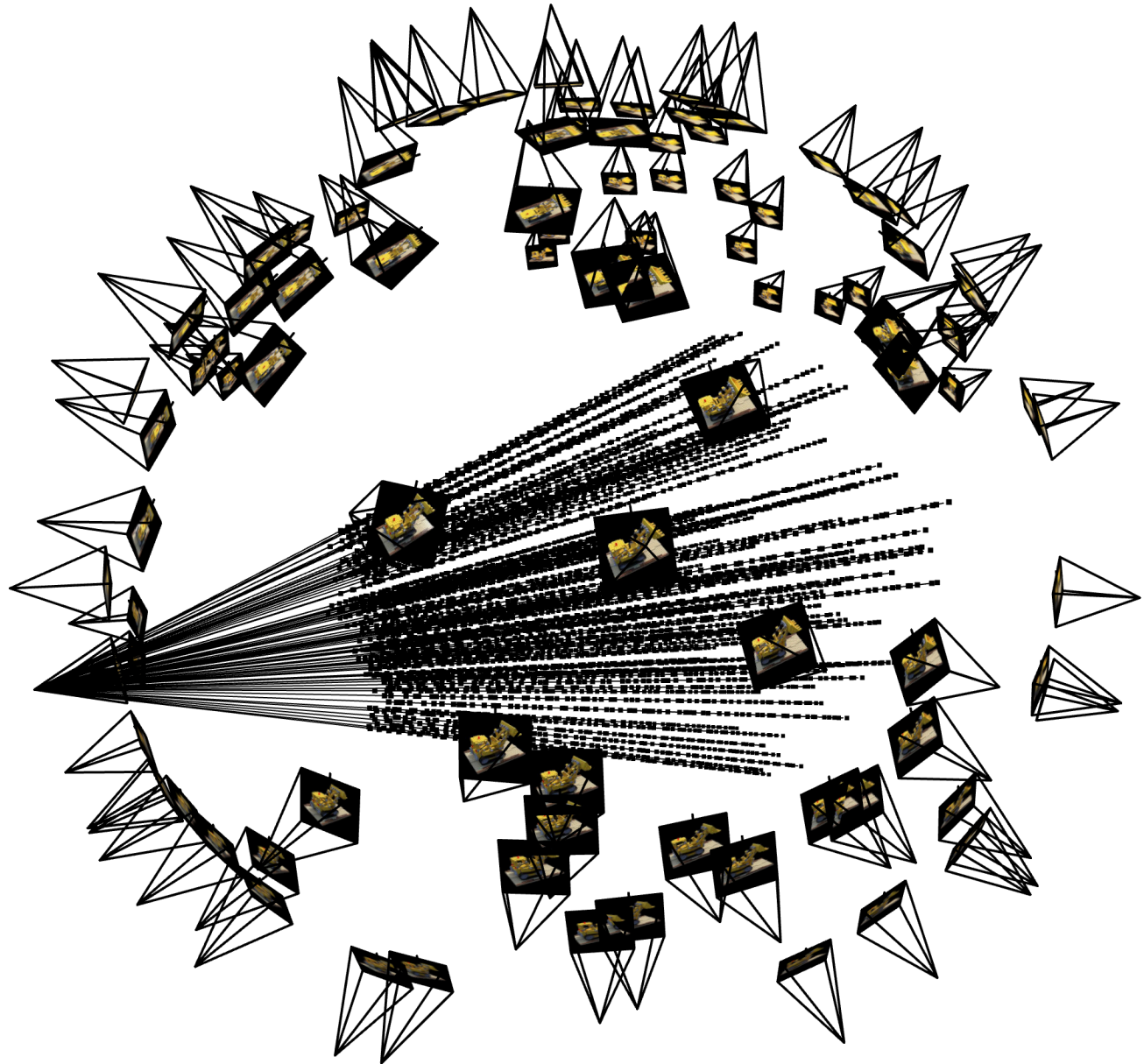
Part 2: 3D Neural Radiance Field

Part 2.1: Create Rays from Cameras

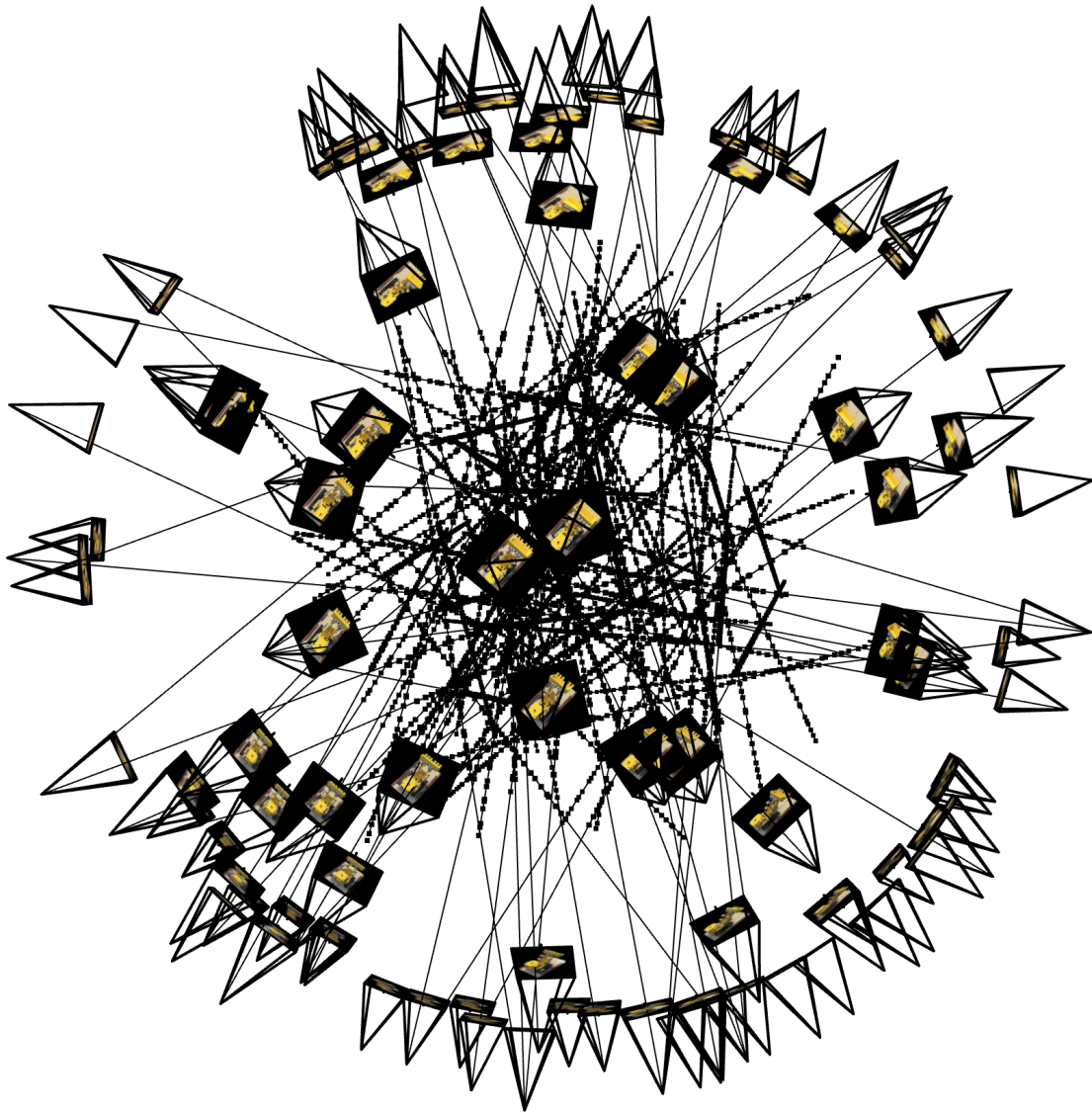
I developed three core transformation functions to generate rays from camera parameters. The **transform** function applies camera-to-world matrices (obtained from Part 0.3) to convert points from camera space to world coordinates. The **pixel_to_camera** function uses the intrinsic matrix from camera calibration (Part 0.1) to map 2D pixel coordinates and depth values into 3D camera space by inverting the projection. Finally, **pixel_to_ray** combines these transformations to generate rays: the ray origin is extracted from the camera-to-world matrix's translation component, while the ray direction is computed by projecting the pixel to world space at unit depth, subtracting the origin, and normalizing.

Part 2.2: Sampling

I implemented a two-stage sampling approach for efficient training. For ray sampling, I created a dataloader that randomly selects N rays per iteration by flattening all pixels across training images, ensuring diverse coverage of the entire dataset. For point sampling along each ray, I uniformly sample 64 positions and add random jitter during training to prevent overfitting to fixed locations. The 3D coordinates are calculated using the parametric equation: $x = \text{ray_origin} + \text{ray_direction} \times t$, where t represents the sampling distance along the ray.



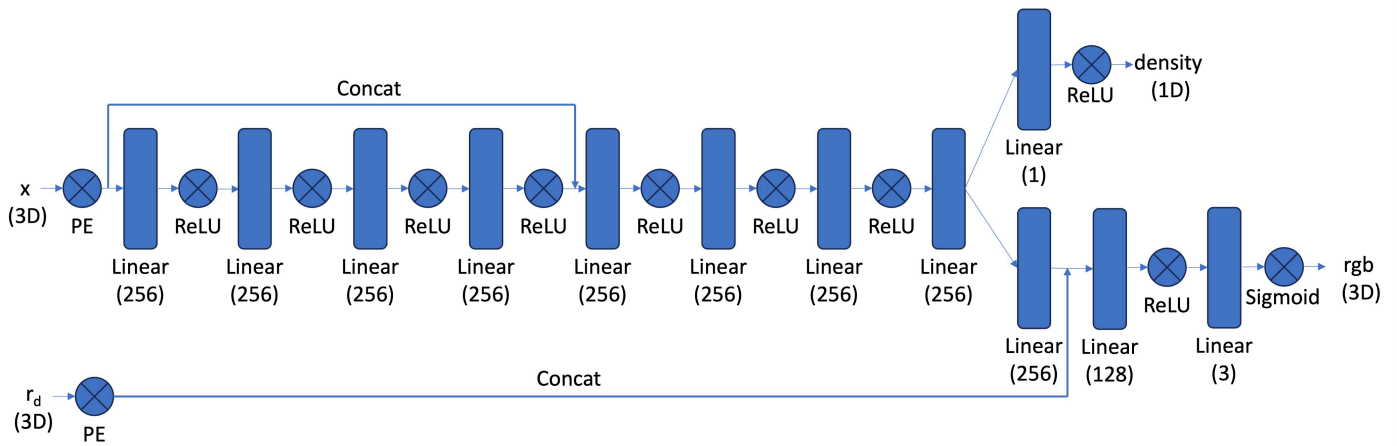
Rays sampled from a single camera position



Rays sampled from multiple camera positions

Part 2.3: Neural Radiance Field

The NeRF model architecture consists of an MLP that takes 3D position $\mathbf{x}$ and viewing direction $\mathbf{r}_d$ as input. Both inputs are encoded with positional encoding (PE) to capture high-frequency details. The network outputs RGB color and volume density σ for each point in 3D space.



NeRF network architecture with positional encoding and skip connections

Part 2.4: Volume Rendering

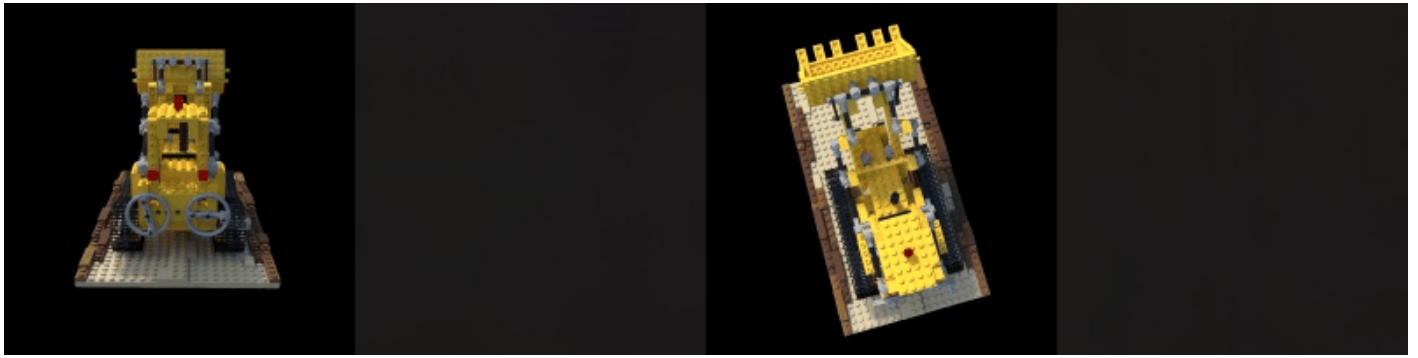
Volume rendering aggregates the colors and densities along each ray to produce the final pixel color. The rendering equation computes the expected color along ray $\mathbf{r}$:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^N T_i (1 - \exp(-\sigma_i \delta_i)) \mathbf{c}_i, \text{ where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j \delta_j\right)$$

where $\mathbf{T}_i$ represents the accumulated transmittance (probability that light reaches point i without being absorbed), $(1 - \exp(-\sigma_i \delta_i))$ is the opacity at point i , $\mathbf{c}_i$ is the RGB color, and δ_i is the distance between adjacent samples. This formulation allows the network to learn volumetric representations of 3D scenes through differentiable rendering.

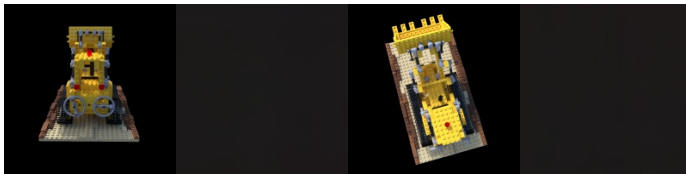
Training on Lego Scene

The NeRF network learns to represent the 3D scene through volume rendering. Below shows the training progression over 5000 iterations.

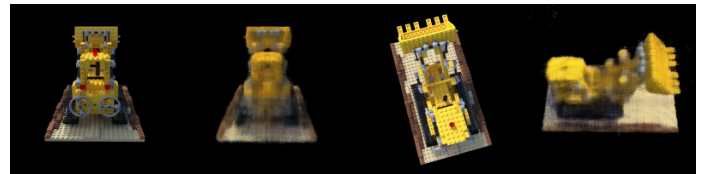


Animated training progression for Lego scene.

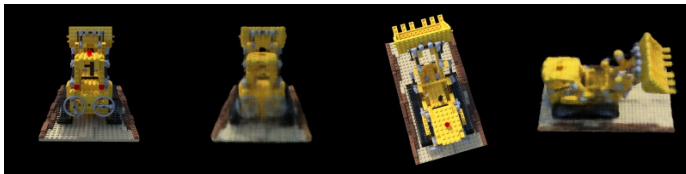
Each frame: [Train GT / Recon] [Validation GT / Recon]



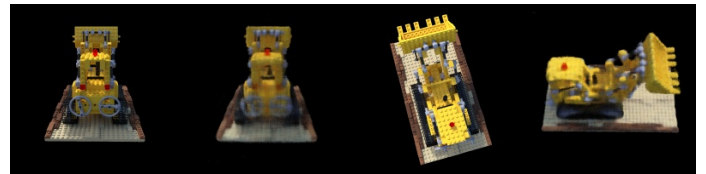
Iteration 0 - [Train GT / Recon] [Validation GT / Recon]



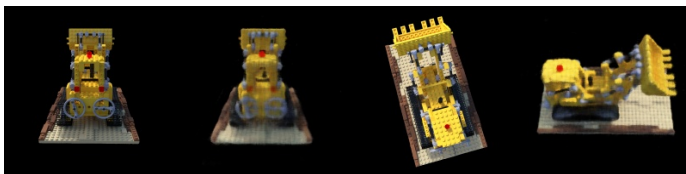
Iteration 1000 - [Train GT / Recon] [Validation GT / Recon]



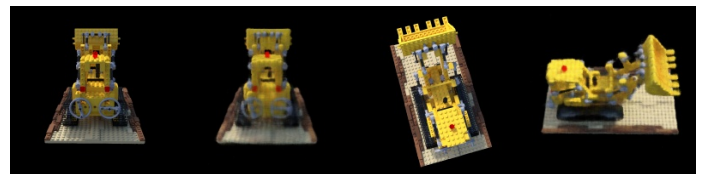
Iteration 2000 - [Train GT / Recon] [Validation GT / Recon]



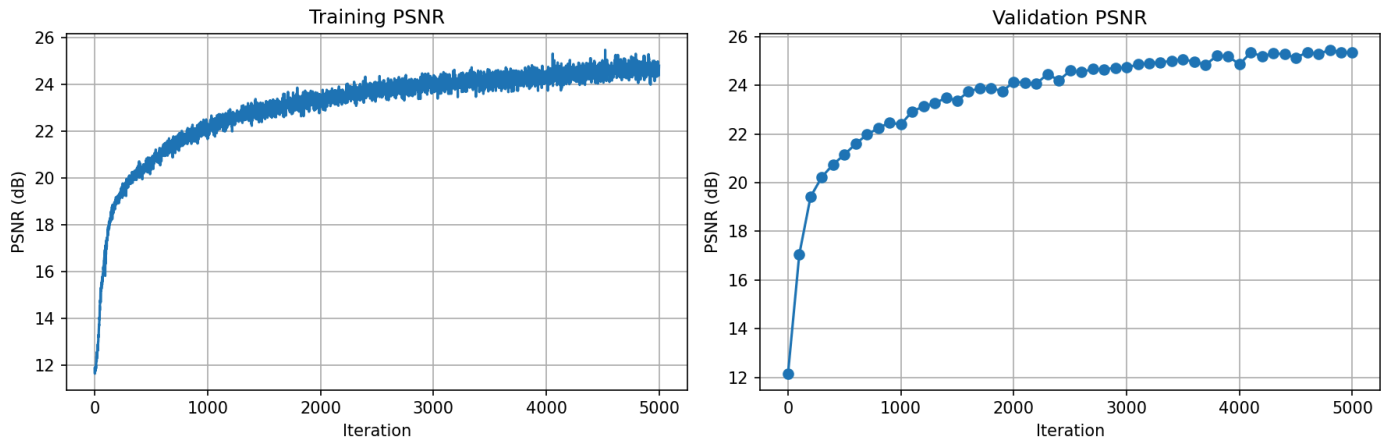
Iteration 3000 - [Train GT / Recon] [Validation GT / Recon]



Iteration 4000 - [Train GT / Recon] [Validation GT / Recon]



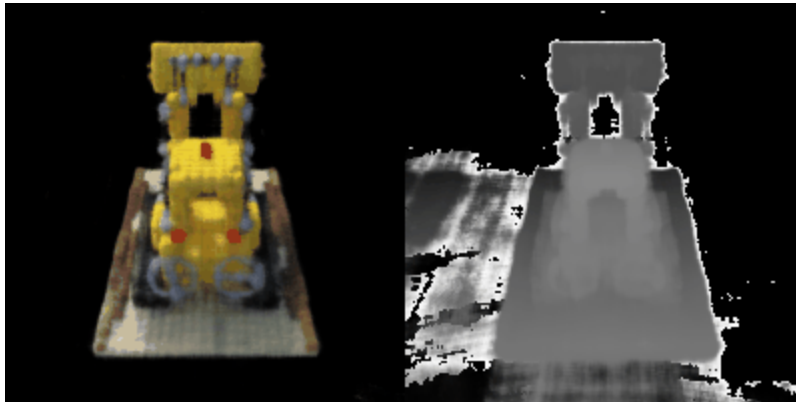
Iteration 4999 - [Train GT / Recon] [Validation GT / Recon]



Validation PSNR curve over training iterations

Novel View Synthesis - Lego (Bells and Whistles: Depth Map)

I implemented spherical rendering with both RGB and depth map visualization. The depth map provides geometric understanding of the 3D scene structure.

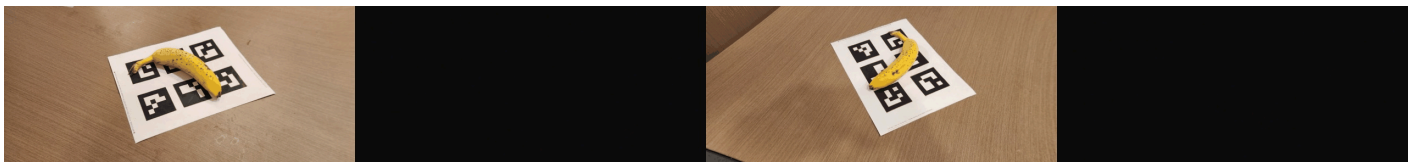


Spherical camera path showing both RGB rendering and depth map visualization

Part 2.6: Custom Object - Banana

Training Results

For my custom object (banana), I adjusted near/far parameters and used appropriate sampling settings. The training was performed over 5000 iterations with similar configurations as the Lego scene.



Animated training progression for banana scene.
Each frame: [Train GT / Recon] [Validation GT / Recon]



Iteration 0 - [Train GT / Recon] [Validation GT / Recon]



Iteration 1000 - [Train GT / Recon] [Validation GT / Recon]



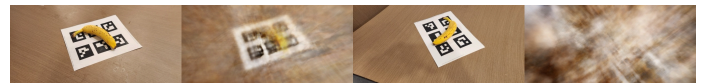
Iteration 2000 - [Train GT / Recon] [Validation GT / Recon]



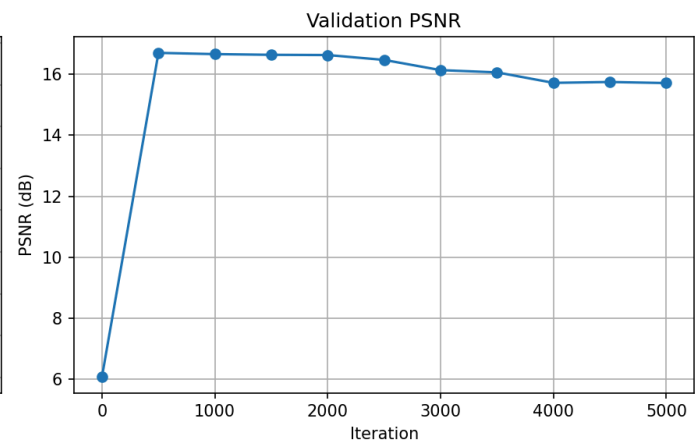
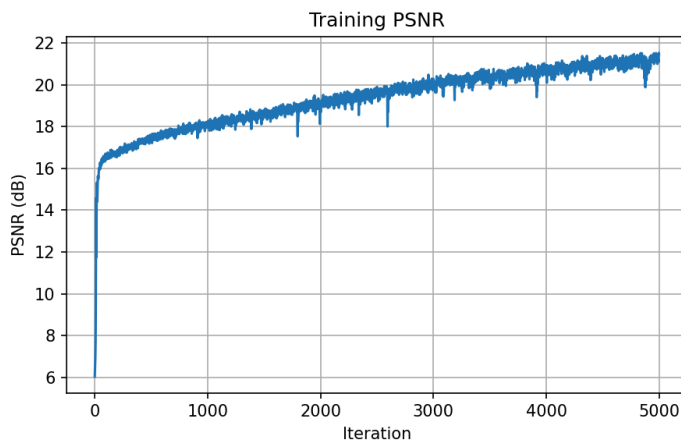
Iteration 3000 - [Train GT / Recon] [Validation GT / Recon]



Iteration 4000 - [Train GT / Recon] [Validation GT / Recon]



Iteration 4999 - [Train GT / Recon] [Validation GT / Recon]



PSNR curves over training iterations for banana scene

Analysis and Observations

During training, the network learned to reconstruct training camera views well, but struggled with validation views. The validation images remained blurry, capturing only faint ArUco tag shapes before failing to converge. This appears to be caused by sparse camera poses in the training set (see Part 0's Camera Pose Visualization). In particular, there were insufficient camera poses from top-down viewing angles. Capturing more diverse data could have improved results, but time constraints prevented recapturing after dismantling the setup.

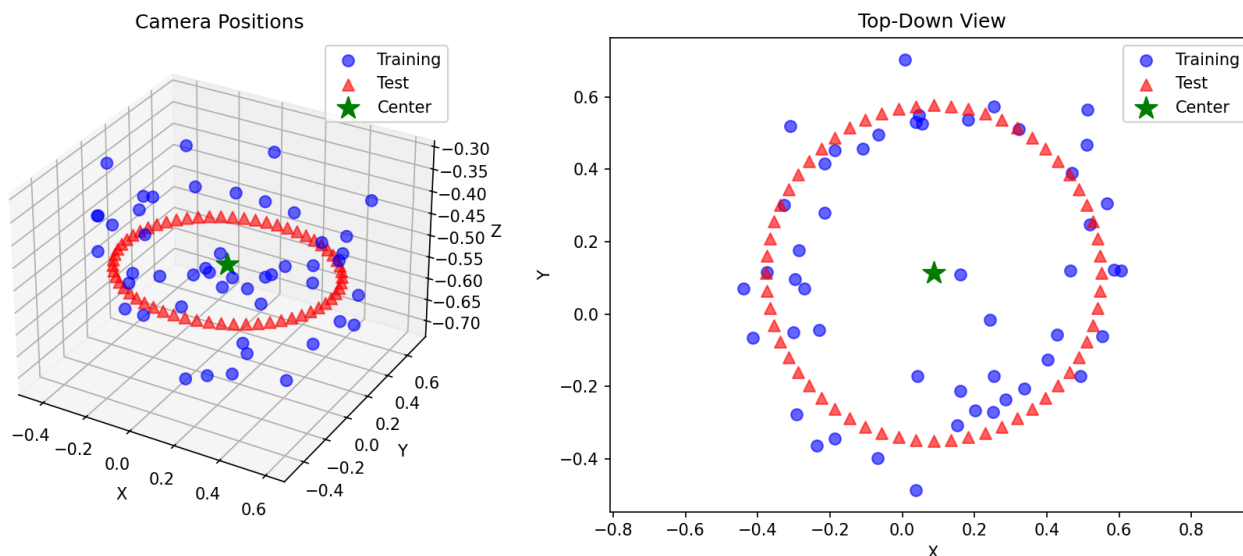
To maximize performance with limited data, I experimented with various hyperparameters including n_{near} , n_{far} , batch size (number of rays), samples per ray, learning rate, learning rate schedule, and network architecture. While these changes did not dramatically improve performance, some adjustments proved helpful:

- Setting $n_{\text{near}}=0.1$ and $n_{\text{far}}=1.0$ after careful calibration using viser visualization
- Reducing network architecture complexity by decreasing the positional encoding frequency for ray direction from $N_{\text{dir}}=6$ to $N_{\text{dir}}=4$

These modifications helped the network progress from completely failing on validation images to at least capturing basic shapes.

Novel View Synthesis Attempts

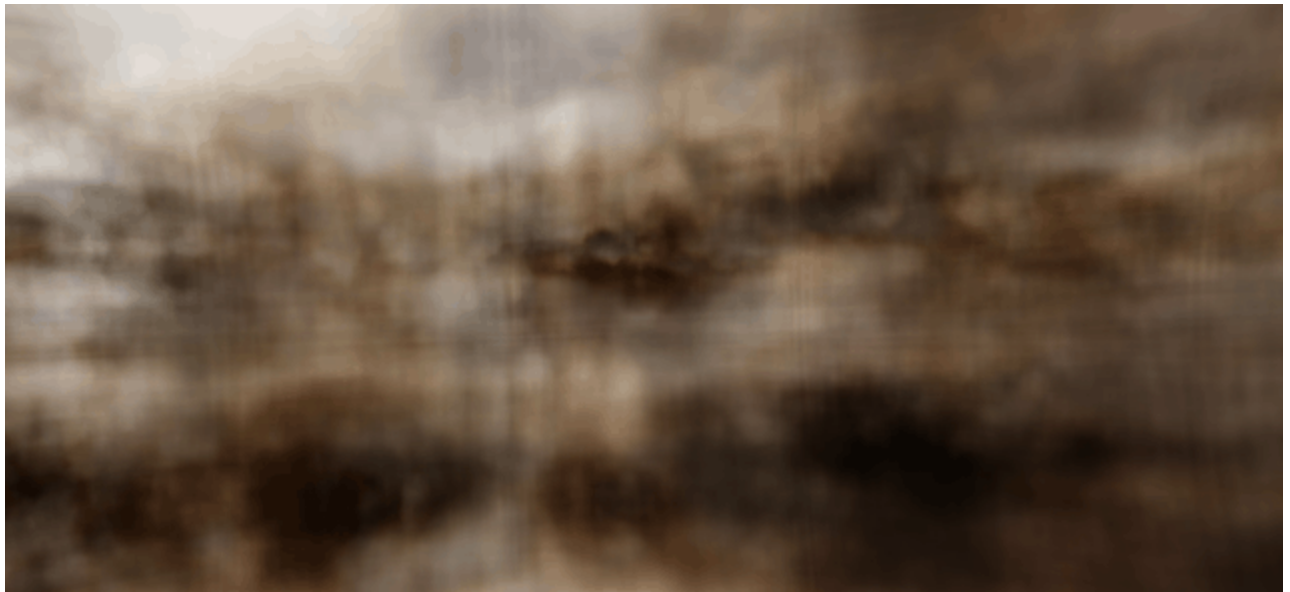
Since the network struggled to learn validation images well, I attempted to find test camera pose trajectories that would effectively cover novel views between training camera poses. Despite trying various camera path configurations, achieving satisfactory novel view synthesis remained challenging due to the sparse training data.



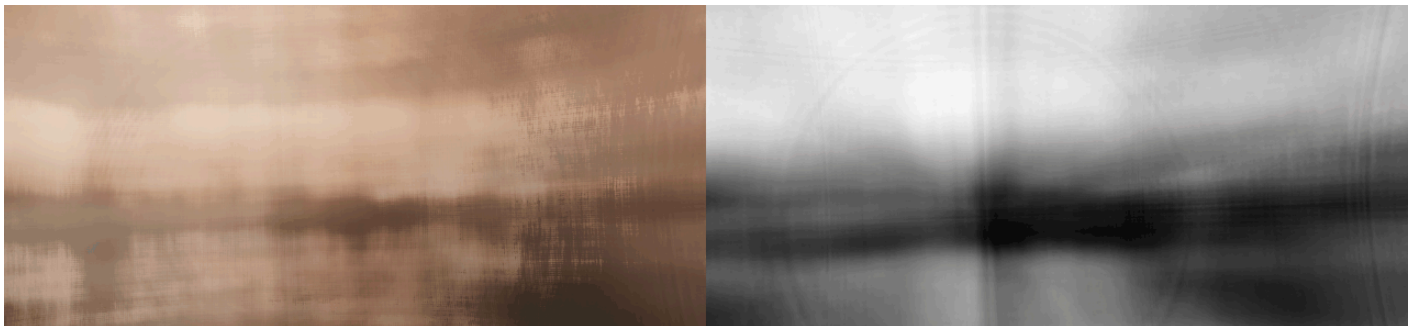
Test camera pose trajectory visualization showing different attempted paths



Novel view synthesis attempt 1



Novel view synthesis attempt 2



Novel view synthesis attempt 3 (at 1000 iterations)

[Back to Home](#)